# Lab session 0

UNIST

Ulsan National Institute of Science and Technology

Mar 11, 2026

# Table of Contents

# Announcements

## Attendance

Please make sure to use mobile attendence!!! According to school regulations, attendance is required for at least three-quarters of classes. In other words, if you absent more than quarter of the classes, you will fail the course.

I know mobile attendance sometimes not work, so if you have any issues with it, please send a email to me (anyunpyo@unist.ac.kr) and carbon copy (CC) to the professor (jooyeon.kim@unist.ac.kr). With title as "*[ITP11701] Mobile Attendance Issue - Your Name*". Please include the screenshot of the error message you get when you try to submit mobile attendance. **Please title as above and CC to the professor, otherwise I might miss your email.**

Feedback on the mobile attendance system in response to attendance requests will be conducted at **the end of the semester**.

# Announcements

## About using LLMs

There are several LLM based services – GPT, Claude Code, etc. – that can help you with learning. You should (can) use them for assignments, projects. That said, hands-on coding and self-study would be more valuable than relying on LLM answers.

## Individual Work Required

All assignments must be completed individually. Copying code from others will result in a negative score. In this case, the assignment in question will be marked as -1 point. Note that lab sessions account for 50% of the total grade. If there is worry about your submitted code being identical to another student's, adding detailed explanations of the code is one way to address such concerns.

# Calendar for lab sessions

| Week | Date | Topic |
|---|---|---|
| 1 | Mar 4 | No lab session |
| 2 | Mar 11 | **LAB 0: Review python and jupyter notebooks** |
| 3 | Mar 18 | LAB 1: Linear algebra with numpy and GPU |
| 4 | Mar 25 | LAB 2: Linear regression models |
| 5 | Apr 1 | LAB 3: Logistic regression |
| 6 | Apr 8 | LAB 4: Softmax classifications |
| 7 | Apr 15 | LAB 5: Classification models |
| 8 | Apr 22 | Midterm break No lab session |
| 9 | Apr 29 | LAB 6: Deep learning from scratch |
| 10 | May 6 | LAB 7: Normalization techniques |
| 11 | May 13 | LAB 8: Convolutional neural networks |
| 12 | May 20 | LAB 9: More on CNN – ResNet |
| 13 | May 27 | Final project announcement |
| 14 | Jun 3 | Election day No lab session |
| 15 | Jun 10 | LAB 10: Transformers |
| 16 | Jun 17 | Final exam week No lab session |

# After Lab 0 you can...

- Set up Python environment on your local machine and Google Colab
- Write and execute Python code in Jupyter Notebooks
- Use basic Python syntax, data structures and libraries for AI programming
- Understanding Jupyter Notebooks features

# Table of Contents

# Environment Setup

In lab sessions we use **Google Colab** — a free Jupyter Notebook environment with pre-installed libraries. If you have your own GPU, set up a local `uv` environment instead.

- `uv` is a Rust-based Python package & project manager (Astral, 2024).
- Replaces `pip + venv + pip-tools` with a single binary.
- Produces a `uv.lock` for exact reproducibility.

# Environment Setup — Install & Create

## Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh

powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

## Create / Reproduce

```
uv init aip2-labs && cd aip2-labs
uv add numpy matplotlib jupyter line-profiler pandoc
uv run jupyter notebook

uv sync && uv run jupyter notebook
```

# Shell Commands in Notebooks

In Google Colab (or any Jupyter kernel), prefix a line with `!` to run it in the shell.

```
'!' delegates a line to the shell
!python --version
!pip show numpy | grep -E 'Name|Version'

Check GPU availability
!nvidia-smi
```

# Table of Contents

# Types and Variables

Python is **dynamically typed** — you can assign any type without declaration.

```
x    = 42
y    = 3.14
s    = "Hello, AIP2!"
lst  = [1, 2, 3, 4, 5]
flag = True

print(type(x), type(y), type(s))
```

- `type(obj)` returns the runtime type of any object.
- All Python values are objects — including `int`, `bool`, `str`.

# Iterables

## Built-in collections

```
my_list  = [1, 2, 3]
my_tuple = (1, 2, 3)
my_set   = {1, 2, 3}
my_dict  = {"a": 1, "b": 2}
```

## Iteration

```
for item in my_list:
    print(item)

for k, v in my_dict.items():
    print(k, v)


print(my_list[1:3])
```

# Iterables

## Generators — lazy evaluation

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

for number in count_up_to(5):
    print(number)
```

Lazy evaluation allows you to generate values on-the-fly without storing the entire sequence in memory.

# Functions

Defined with `def`; `return` sends a value back.

```
def sign(x):
    if x > 0:
        return "positive"
    elif x < 0:
        return "negative"
    else:
        return "zero"

for num in [10, -5, 0]:
    print(f"{num} is {sign(num)}")
```

- Use f-strings (`f"{expr}"`) for readable formatting.
- Functions are first-class objects — passable as arguments.

# Python Libraries

```
import math
print(math.sqrt(16))
print(math.pi)

import numpy as np
print(np.array([1, 2, 3]))
print(np.pi)
```

Key libraries for this course:

- `numpy` — numerical arrays, linear algebra
- `matplotlib` — data visualisation
- `torch` — deep learning (from Lab 3 onward)
- `line-profiler` — line-by-line profiling

# Table of Contents

# Classes and Objects

Every Python value is an **object**. A **class** is its blueprint.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        return Vector(self.x + other.x,
                      self.y + other.y)

v1 = Vector(3, 4)
v2 = Vector(1, 2)
print(v1 + v2)
```

# Dunder Methods

**Magic (dunder) methods** let your class integrate with Python syntax.

| Method | Triggered by | PyTorch analogy |
|---|---|---|
| `__init__` | `MyClass(...)` | layer definitions |
| `__repr__` | `repr(obj)` | model summary string |
| `__call__` | `obj(x)` | dispatches to `forward(x)` |
| `__len__` | `len(obj)` | `Dataset.__len__` |
| `__getitem__` | `obj[i]` | `Dataset.__getitem__` |
| `__add__` | `a + b` | tensor operator overloading |

Why now? Every PyTorch model subclasses `torch.nn.Module` — the same pattern.

# Inheritance and super()

```
class Module:
    def __call__(self, x):
        return self.forward(x)
    def forward(self, x):
        raise NotImplementedError

class ReLU(Module):
    def forward(self, x):
        return max(0.0, x)

class Sequential(Module):
    def __init__(self, *layers):
        self.layers = layers
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

model = Sequential(ReLU())
print(model(-1.0))
```

# Copy Objects

Common mistake: assignment creates a **reference**, not a copy.

## Shallow copy

```
import copy
lst1 = [1, 2, [3, 4]]
lst2 = lst1.copy()

lst2[0] = 99
lst2[2][0] = 99
```

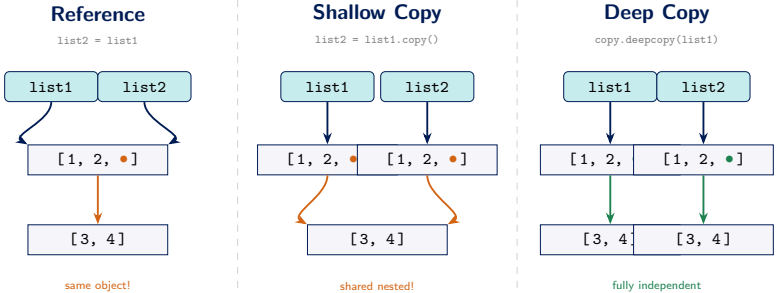
## Deep copy

```
import copy
lst1 = [1, 2, [3, 4]]
lst2 = copy.deepcopy(lst1)

lst2[2][0] = 99
```

Use `copy.deepcopy()` whenever your object contains nested mutable structures.

# Copy Semantics in Memory



# Assert and Try-Except

## assert

```
def add(a, b):
    return a + b

assert add(2, 3) == 5
assert add(-1, 1) == 0
print("PASSED!")
```

Raises `AssertionError` if `False`.

## try-except

```
try:
    t = (1, 2, 3)
    t[0] = 10
except TypeError as e:
    print("Error:", e)
```

Catches exceptions gracefully.

# Table of Contents

# Jupyter Magic Commands

Magic commands extend the IPython kernel. `%` applies to one line; `%%` applies to the entire cell.

| Magic | Scope | Use case |
|---|---|---|
| `%%time` | cell | One-shot wall-clock timing |
| `%%timeit` | cell | Statistical benchmark (mean $\pm$ std) |
| `%debug` | line | Post-mortem debugger after an exception |
| `%prun` | line | `cProfile` on a single statement |
| `%lprun` | line | Line-by-line timing (`line_profiler`) |
| `%who` | line | List all variables in scope |
| `%load_ext` | line | Load an IPython extension |

# Profiling Example

```
%%time
import numpy as np
result = sum([i**2 for i in range(1000000)])
print(f"Result: {result}")
```

```
%%timeit
sum([i**2 for i in range(10000)])
```

```
%load_ext line_profiler
%lprun -f py_matmul py_matmul(A, B)
```

# Table of Contents

# Exercise

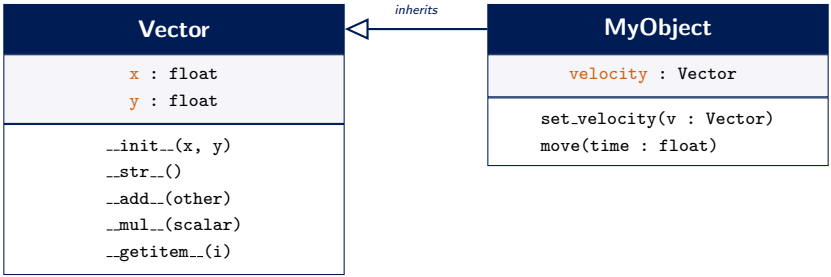
Implement the `TODO`s in `lab0.ipynb`:

1. **Inner product of two Vectors**
   - `inner_product(v1, v2)` $\rightarrow$ $x_1x_2 + y_1y_2$
2. **2D Object dynamics**
   - `MyObject.set_velocity(v)` — store a velocity `Vector`
   - `MyObject.move(time)` — update position: $p \mathrel{+}= v \cdot t$
3. **Fix shallow copy**

## Submission

Export `lab0.ipynb` as PDF and rename it `<student_id>_lab0.pdf`.

# Class Hierarchy: Vector & MyObject



# Summary

- We set up our environment for Python programming and Jupyter Notebooks.
- We reviewed basic Python programming concepts, including variables, data types, control structures, functions, and object-oriented programming.
- We introduced Jupyter Notebooks and how to use magic commands for profiling and debugging.

# Further Resources

- Python Official Tutorial
- Jupyter Documentation
- uv Package Manager

Or several online courses related to Python basics as follow

- Ernest Ryu's Python Tutorial
- Jinseok Seol's 99장으로 살펴보는 파이썬 기초

*END*